

附件（四）

深圳大学实验报告

课程名称： 操作系统

实验项目名称： 内存管理

学院： 计算机与软件学院

专业： 计算机科学与技术（创新班）

指导教师： 周明洋

报告人： 李文俊 学号： 2023150001 班级： 高性能

实验时间： 2026 年 05 月 31 日

实验报告提交时间： 2026 年 05 月 31 日

教务处制

一、实验目的与要求：

- (1) 加深对内存分配与使用操作的直观认识；
- (2) 掌握 Linux 操作系统的内存分配与释放接口，理解 malloc、free、mmap、munmap 与内核虚拟内存区域之间的关系；
- (3) 学会结合 /proc/pid/maps、/proc/pid/status 和 /proc/pid/smaps 观测进程逻辑地址、虚拟空间和物理页占用的变化；
- (4) 通过读触页、写触页、双进程竞争和访问局部性对比，验证按需分页、页面置换与工作集局部性对系统性能的影响。

二、方法、步骤：(说明程序相关的算法原理或知识内容，程序设计的思路和方法，可以用流程图表述，程序主要数据结构的设计、主要函数之间的调用关系等)

1. 核心知识点

1.1 进程虚拟地址空间与 VMA 组织

在 Linux 中，每个进程通过 `task_struct->mm_struct` 描述自己的用户态虚拟地址空间，并以一组 `struct vm_area_struct` (VMA) 表示其内部的连续区间。代码段、数据段、堆、动态库.so 映射、栈、vvar 和 vdso 等区域都各自对应一个 VMA，由内核按权限和文件来源加以区分。

`/proc/<pid>/maps` 正是这些 VMA 按地址顺序的线性化展示，每一行依次给出起止地址、权限位 (rwxp)、文件偏移、设备号、inode 和映射的文件名，匿名映射对应的文件名通常为 `空` 或显示为 `[heap]`、`[stack]`、`[anon]`。本实验中 `vm_layout.c` 使用 `malloc` 连续申请六个 128MB 大块，`glibc` 把这些块走匿名 `mmap` 路径，因此六块虚拟区在内核眼中可以被合并成一段连续的 `rw-p` 匿名 VMA；随后 `free` 第 2、3、5 号触发 VMA 拆分，`maps` 中出现 128MB 与 256MB 两类空洞，正是 VMA 链路上“区间分裂、空洞保留”的直接证据。

1.2 malloc、free、mmap 与 munmap 的分工

`malloc` 只是 `glibc` 提供的 C 库接口，真正改变进程地址空间的是底层系统调用 `brk/sbrk` 与 `mmap`。

- (1) 对于小于 `MMAP_THRESHOLD` (默认 128KB) 的请求，`glibc` 从堆顶通过 `brk` 扩展或从已有 `arena` 的 `free list` 中切分；
- (2) 对于本实验中的 128MB、1024MB 大块请求，`glibc` 直接调用 `mmap` 向内核申请独立的匿名 VMA。`free` 针对大块时 `glibc` 会调用 `munmap`，把整段 VMA 从 `mm_struct` 的 `vma` 链上摘除，因此 `vm_layout.c` 中 `free(p[1])`、`free(p[2])`、`free(p[4])` 看到的不是堆指针后移，而是 `maps` 中对应区段的消失或被切分。
- (3) `touch_pages.c` 与 `locality_compare.c` 则直接调用 `mmap` 以 `PROT_READ|PROT_WRITE + MAP_ANONYMOUS` 分配，从而避开 `glibc` 的 `arena` 管理，直接观察 VMA 与缺页行为。

1.3 /proc 状态字段与 smaps 解释

/proc/<pid>/status 以 key:value 的简洁形式呈现进程级内存摘要：

- (1) **VmSize** 为整个用户态虚拟地址空间总和 (KB)
- (2) **VmData** 反映堆、匿名映射和数据段的总虚拟规模
- (3) **VmRSS** 为当前驻留于物理内存的页面总量
- (4) **RssAnon**、**RssFile**、**RssShmem** 分别给出匿名页、文件映射页、共享内存页的驻留量
- (5) **VmSwap** 则统计已被换出到交换区的匿名页大小
- (6) `vm_layout.c` 中 `print_status_summary` 函数通过 `fopen("/proc/getpid()/status")` 按字段前缀匹配输出，正是为了在每次 `malloc/free` 前后稳定捕获这五个量的变化。

/proc/<pid>/smaps 把同样的统计拆分到每个 VMA 下，进一步暴露 Pss (按引用进程数加权后的实际占用)、`Shared_Clean/Private_Clean`、`Anonymous`、`Swap` 等字段，可用于区分共享库与私有匿名页对内存的真实贡献。

1.4 页表、4KB 页与缺页异常

虚拟地址并不直接寻址物理 RAM，CPU 通过多级页表把虚拟页号 (VPN) 翻译为物理页框号 (PFN)，再合上页内偏移得到物理地址；x86-64 常见为 4 级页表 (PGD/PUD/PMD/PTE)。只有当对应 PTE 中的 `Present` 位置 1 时 CPU 才能直接命中 TLB 或表项；若 PTE 缺失或权限不足，硬件触发缺页异常，控制流陷入内核 `do_page_fault`。本实验中所有遍历步长固定为 `sysconf(_SC_PAGESIZE)` (即 4096 字节)，目的是让每一次访问都恰好跨入一个新页，从而把每次循环的开销转化为一次明确的缺页事件。依据触发原因，缺页可分为：`minor fault` (次缺页，物理页已在内存中，只需补 PTE)、`major fault` (主缺页，需要从 swap 或文件中读入物理页) 和 `copy-on-write fault`；`touch_pages` 与 `locality_compare` 都通过 `getrusage` 采集这两个计数。

1.5 按需分页、共享零页与写时分配

`mmap` 返回成功只代表内核已经为 VMA 登记好虚拟区间，PTE 此时仍是空的；直到进程首次访问该 VMA 中的某一页，内核才在缺页路径上为其分配物理页框，此即按需分页 (demand paging)。对匿名页的首次“只读”访问，内核会把所有未写入的虚拟页临时映射到同一个全局只读的零页 (`ZERO_PAGE`)，PTE 仅指向这一页，因此 `touch_pages` 的 `read` 模式下 `VmSize` 会迅速扩张但 `RssAnon` 几乎不变，且 `minor faults` 计数虽高却没有真实的物理分配。当进程首次“写”到该匿名页时，处理器检测到只读 PTE 上的写操作触发写时复制 (`copy-on-write`) 逻辑——内核为该虚拟页分配新的私有物理页，把零页内容拷过去并把 PTE 改为可写，随后 `RssAnon` 会以 4KB 粒度逐页上升，这正是写触页实验中 `VmRSS` 显著拔高的微观机制。

1.6 overcommit、RLIMIT_AS 与最大虚拟映射

64 位进程的用户态虚拟地址空间上限远大于物理内存，Linux 内核因此可以对未实际触页的匿名映射进行内存超量承诺 (overcommit)，由 `/proc/sys/vm/overcommit_memory` 控制具体策略。`max_vmem.c` 调用 `mmap` 时传入 `PROT_NONE` 和 `MAP_NORESERVE`：`PROT_NONE` 让该区间不可读写，避免任何意外访问触发缺页；`MAP_NORESERVE` 显式告诉内核不要为该映射预留 swap 额度，从而只测试虚拟地址空间能否保留而不强制兑现物理页。`try_map` 将每个候选长度的 `mmap` 与立即 `munmap` 配对，保证测试本身不会占用累积地址空间。若

getrlimit(RLIMIT_AS, &lim)显示 RLIM_INFINITY, 则进程级地址空间没有上限, 实测最大值由 64 位用户地址空间布局 (如 aarch64 上的 TASK_SIZE) 与内核 VMA 管理决定, 因此实验在 Ubuntu 虚拟机上能测到约 170TiB, 远高于 3.8GiB 物理内存。

1.7 页面置换、swap 与物理页竞争

当多个进程的匿名工作集之和接近或超过可用物理内存时, 内核内存回收路径 (kswapd 后台线程或直接回收路径 direct reclaim) 会扫描 LRU 链表, 选择不活跃匿名页通过 swap 子系统写到交换分区, 并回收其页框分配给当前急需内存的进程。双进程竞争实验中 A 进程先以 touch_pages 1792 write-hold 方式写入 1792MB 并保持映射, 使其匿名页全部兑现; B 进程随后写入 4096MB 造成内存压力, 内核优先把 A 中"不活跃"的私有匿名页写到 swap。从外部读取/proc/A/status 可以看到 A 的 VmRSS 由约 1.79GB 回落到约 1MB, 而 VmSwap 同步攀升到约 1.75GB——A 的虚拟地址区间没有变, 只是 PTE 被改写为 swap entry, 对应物理页框已被回收并交给 B 使用, 这就是物理页竞争的内核侧表现。

1.8 访问局部性与工作集大小

工作集模型 (Working Set Model) 指出在任意时间窗口 Δ 内, 进程实际访问的页面集合 $W(t, \Delta)$ 决定其常驻物理内存所需的最小页框数; 若 $W(t, \Delta)$ 大于可分配的物理页框数, 进程就会陷入颠簸 (thrashing)。locality_compare.c 使用相同访问总量但不同顺序对比这一现象: access_global 把外层 repeat 循环放在最外面 (for r $\rightarrow$ for page), 相邻两次访问同一页之间间隔整个 pages 页扫描, 此时 W 窗口约等于整个 4GB 映射, 远超物理内存; access_group16 将 16 个连续页固定为一组 (W 约 64KB), 组内先重复 repeat 遍再前进到下一组, 使工作集长期保持小于一个 TLB 条目+少量物理页。由于 access_global 的下一访问命中目标页时多数页已被换出, 主缺页 (major fault) 激增、运行时间被 swap I/O 拖长; 而 access_group16 几乎不产生主缺页, 运行时间下降一个数量级, 给出了"页面置换性能取决于工作集大小而非访问总量"的可测证据。

2. 程序设计思路

2.1 程序整体模块划分

本次实验由四个职责单一的可执行程序组成, 分别负责虚存布局观测、最大虚拟映射测量、读写触页对比与访问局部性对比, 所有现象都用独立程序承载、各自只输出与自身主题相关的状态, 便于报告分小节阐述与读者重复运行。四个程序的具体设计思路依次在 2.2~2.5 节展开, 本节只给出宏观分工。

- (1) vm_layout.c 负责连续申请六个 128MB 块、释放 2 号、3 号、5 号块、再申请 1024MB 和 64MB, 并在每一步前后复制/proc/pid/maps 与/proc/pid/status
- (2) max_vmem.c 使用 MAP_NORESERVE 和 PROT_NONE 执行单次连续匿名映射的指数扩张与二分测试, 用于测量虚拟地址空间边界
- (3) touch_pages.c 统一完成读触页、写触页和保持映射三种模式, 用于对照 VmRSS 与 VmSwap 变化
- (4) locality_compare.c 固定总访问次数, 分别执行全空间逐页循环和 16 页分组访问, 用 getrusage 记录主缺页次数和运行时间。

2.2 vm_layout.c: 虚存布局观测程序的设计思路

- (1) 入口先建立 snapshots 输出目录并打印自身 pid, 使后续每个状态文件与外部 /proc 观察都能落到同一进程上下文。
- (2) 以 128MB 为单位连续 malloc 六个块, 每个 malloc 调用前后各做一次 snapshot, 把当时的 VmSize、VmRSS、VmData 与完整 maps 一并抄存到磁盘。
- (3) 按 2 号、3 号、5 号的顺序逐个 free 这三块, 每次 free 前后再次 snapshot, 使后续可以直观对比 VMA 在拆分前后的拓扑变化。
- (4) 申请 1024MB 大块, 记录该块落在低地址区域并与 6 号块合并的现象, 验证大请求无法塞进已释放空洞的判断。
- (5) 再申请 64MB 小块, 重点观察其是否被安排进 2/3 号释放后留下的空洞, 并把实测地址打印出来供报告进行预测验证。
- (6) 退出前统一 free 全部仍持有的块并调用 malloc_trim(0), 最后做一次 cleanup snapshot, 让残留 arena 信息也留在快照里以便复盘。

2.3 max_vmem.c: 最大虚拟映射测量程序的设计思路

- (1) 入口先调用 getrlimit(RLIMIT_AS) 打印进程级地址空间软上限, 判断是否启用了 ulimit 级人为限制。
- (2) 把 mmap+munmap 封装为 try_map:成功立即归还, 避免测试过程本身占用大量地址空间; mmap 参数固定为 PROT_NONE 与 MAP_NORESERVE, 只测能否保留虚拟区间。
- (3) 从 1GiB 起按 2 的幂指数翻倍寻找首个失败长度, 把 (low, high) 上下界确定下来, 避免线性试探太慢。
- (4) 在 (low, high) 区间按 sysconf(_SC_PAGESIZE) 对齐做二分搜索, 每步收敛精度为一个物理页, 使结果与内核管理粒度一致。
- (5) 输出最终最大单次连续匿名虚拟映射长度, 同时给出 GiB 与 TiB 两种单位, 便于与物理内存数量级做直观对比。

2.4 touch_pages.c: 读写触页观测程序的设计思路

- (1) 把映射大小 (MB)、访问模式 (read / write / write-hold) 与保持秒数都做成命令行参数, 让同一份代码可以覆盖多种触页实验。
- (2) mmap 调用前后各打印一次 status, 证明“虚拟区间已建立但物理页尚未兑现”时 VmSize 与 VmRSS 会显著分离。
- (3) 按 sysconf(_SC_PAGESIZE) 返回的页大小作为步长循环遍历整个映射, 让每次循环恰好跨入一个新页, 使每次访问对应一次明确的页级事件。
- (4) read 分支用 volatile sink 做异或读取, 防止编译器把整段循环优化掉; write / write-hold 分支对每个页执行原地 +1, 强制走 copy-on-write 路径分配私有物理页。
- (5) 循环前后通过 getrusage 采集 minor/major 缺页增量, 并再次读取 status, 把 RssAnon 与 VmSwap 与缺页计数对照分析。
- (6) write-hold 模式在触页后 sleep(hold_seconds), 保持映射活跃, 专门用于双进程竞争实验里供另一终

端读取本进程的 `/proc/pid/status`。

2.5 locality_compare.c: 访问局部性对比程序的设计思路

- (1) 把映射大小 (MB)、重复轮数 repeat、访问模式 (global / group16) 都做成命令行参数, 保证两种模式的访问总量严格相等, 差异只来自顺序。
- (2) 一次性 mmap 出足够大的匿名映射 (实验取 4096MB, 超过虚拟机 3.8GiB 物理内存), 让页面置换压力真正进入测量范围。
- (3) global 模式把 repeat 放在最外层: 每一轮先扫完整 pages 个页才回头, 下一次访问同一页时间间隔约等于整个工作集大小。
- (4) group16 模式把 16 个连续页固定为一组, 组内先完成全部 repeat 轮再前进到下一组, 使瞬时工作集长期保持在约 64KB 量级。
- (5) 两种模式都执行 `buf[page * page_size]++` 触发写, 确保每次访问都对应一次页级 W 事件, 并复用同一计时与 getrusage 框架。
- (6) 循环结束后输出 elapsed seconds、minor faults、major faults、max RSS 四个标量, 把"访问顺序对换页代价的影响"量化到可直接对比的数字。

2.6 关键约束与防错机制

- (1) 大块虚拟内存测试使用 PROT_NONE 和 MAP_NORESERVE, 目的是测虚拟地址可映射边界, 而不是无意义地把物理内存写爆。
- (2) 读写触页实验把页面步长固定为 sysconf(_SC_PAGESIZE)返回的 4KB, 使每一次访问都对应一个明确的页级事件, 避免把缓存行或字节级循环误当成页分配证据。
- (3) 双进程竞争实验不依赖 A 进程自身日志, 而是在 B 进程运行前后由外部读取 `/proc/<pid>/status`, 避免 stdout 被重定向后落入全缓冲影响实时观察。

3. 流程图表示

本实验整体流程按"环境部署→虚存布局观测→最大虚拟映射测量→读触页与写触页对比→双进程竞争→访问局部性对比"的顺序展开, 覆盖虚拟内存生命周期的申请、兑现、置换与回收四个阶段。

虚拟内存分配和复用流程图:

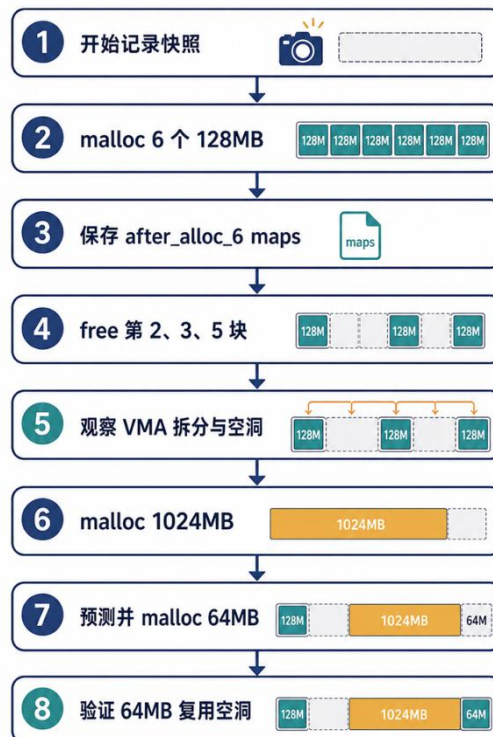


图 1 虚拟内存分配和复用流程图

最大虚拟空间测试流程图：



图 2 最大虚拟空间测试流程图

读触页和写触页对比流程图：



图 3 读触页和写触页对比流程图

双进程竞争物理页流程图:



图 4 双进程竞争物理页流程图

访问局部性对比流程图:



图 5 访问局部性对比流程图

三. 实验过程及内容: (对程序代码进行说明和分析, 越详细越好, 代码排版要整齐, 可读性要高)

1. 实验编译与运行环境部署

1.1 工作目录与编译命令

宿主机和 Ubuntu 虚拟机中均建立 exp3 目录, 虚拟机实际工作目录为/home/wenjun/exp3。编译命令如下:

```
gcc -Wall -Wextra -O0 -g -o vm_layout vm_layout.c
```

```
gcc -Wall -Wextra -O2 -g -o max_vmem max_vmem.c
```

```
gcc -Wall -Wextra -O2 -g -o touch_pages touch_pages.c
```

```
gcc -Wall -Wextra -O2 -g -o locality_compare locality_compare.c
```

其中 vm_layout 关闭优化是为了让观测顺序和源码顺序保持直观, 其余三个程序开启 O2 是为了减少循环本身的额外开销, 使耗时差异更集中地反映内存系统行为。

```
wenjun@2023150001: ~/exp3
wenjun@2023150001:~/exp3$ cd ~/exp3
wenjun@2023150001:~/exp3$ pwd
/home/wenjun/exp3
wenjun@2023150001:~/exp3$ uname -a
Linux 2023150001 5.4.0-216-generic #236-Ubuntu SMP Fri Apr 11 19:55:34 UTC 2025
aarch64 aarch64 aarch64 GNU/Linux
wenjun@2023150001:~/exp3$ getconf PAGE_SIZE
4096
wenjun@2023150001:~/exp3$ free -h
              total        used        free      shared  buff/cache   available
Mem:           3.8Gi        166Mi        2.9Gi         1.0Mi        773Mi        3.5Gi
Swap:          3.8Gi          40Mi        3.8Gi
wenjun@2023150001:~/exp3$ ls -l vm_layout.c max_vmem.c touch_pages.c locality_co
mpare.c
locality_compare.c
max_vmem.c
touch_pages.c
vm_layout.c
wenjun@2023150001:~/exp3$
```

图 6 Ubuntu 虚拟机实验环境与工作目录终端页面

2. 虚拟内存布局与空洞复用实验

2.1 快照记录与分配释放代码

vm_layout.c 的核心思想是在每个分配与释放动作前后调用 **snapshot**：snapshot 内部读取 /proc/getpid()/status 中的 VmSize、VmRSS、VmData 等字段，并把完整 maps 复制到 snapshots 目录。这样每个动作都有可回溯的状态文件，而不是只依赖终端瞬时输出。

核心代码如下：

```
static void snapshot(const char *label) {
    ++snapshot_id;
    printf("\n==== snapshot %02d: %s =====\n", snapshot_id, label);
    print_status_summary();
    copy_proc_file("status", label);
    copy_proc_file("maps", label);
}

for (int i = 0; i < BLOCKS; ++i) {
    snapshot("before_alloc");
    p[i] = malloc(128 * MB);
    memset(p[i], 0, 4096);
    snapshot("after_alloc");
}

int free_list[] = {2, 3, 5};
for (size_t i = 0; i < 3; ++i) {
    int idx = free_list[i] - 1;
    free(p[idx]);
    p[idx] = NULL;
    snapshot("after_free");
}
```

图 7 vm_layout 核心 snapshot 函数代码

这段代码的关键不在 malloc 本身，而在 snapshot 插入点。snapshot 在每次动作前后读取同一个进程的 status 和 maps，保证 VmSize、VmRSS、VmData 与 VMA 地址变化能够一一对应。p 数组保留六个块的首地址，使后续 free 能够精确释放 2 号、3 号、5 号；

细节：每次 malloc 后只写首个 4KB 页，是为了触发最小限度的页表建立，同时避免把 768MB 空间全部兑现为物理页。free_list 中的编号先减 1 再访问数组，保证实验报告中的 1 号到 6 号与 C 语言数组下标之间不会错位。

2.2 maps 片段与 64MB 落点分析

```
wenjun@2023150001: ~/exp3
wenjun@2023150001:~/exp3$ echo 'Saved /proc maps snapshots: anonymous VMAs'
Saved /proc maps snapshots: anonymous VMAs
wenjun@2023150001:~/exp3$ grep '^ffff5b4ba' snapshots/13_after_alloc_6.maps
ffff5b4ba000-ffff8b4c0000 rw-p 00000000 00:00 0
wenjun@2023150001:~/exp3$ grep -E '^ffff(5b4ba|6b4bc|834bf)' snapshots/19_after_free_5.maps
ffff5b4ba000-ffff634bb000 rw-p 00000000 00:00 0
ffff6b4bc000-ffff734bd000 rw-p 00000000 00:00 0
ffff834bf000-ffff8b4c0000 rw-p 00000000 00:00 0
wenjun@2023150001:~/exp3$ grep -E '^ffff(1b4b9|6b4bc|834bf)' snapshots/21_after_alloc_1024.maps
ffff1b4b9000-ffff634bb000 rw-p 00000000 00:00 0
ffff6b4bc000-ffff734bd000 rw-p 00000000 00:00 0
ffff834bf000-ffff8b4c0000 rw-p 00000000 00:00 0
wenjun@2023150001:~/exp3$ grep -E '^ffff(1b4b9|6b4bc|7f4be)' snapshots/23_after_alloc_64.maps
ffff1b4b9000-ffff634bb000 rw-p 00000000 00:00 0
ffff6b4bc000-ffff734bd000 rw-p 00000000 00:00 0
ffff7f4be000-ffff8b4c0000 rw-p 00000000 00:00 0
wenjun@2023150001:~/exp3$
```

图 8 终端查看 maps 关键匿名映射片段

/proc/pid/maps关键匿名映射片段

after_alloc_6: 六个128MB块连续合并
ffff5b4ba000-ffff8b4c0000 768.0 MB

after_free_5: 释放2、3、5号后留下空洞
ffff5b4ba000-ffff634bb000 128.0 MB
ffff6b4bc000-ffff734bd000 128.0 MB
ffff834bf000-ffff8b4c0000 128.0 MB

after_alloc_1024: 1024MB向低地址扩展并与6号块合并
ffff1b4b9000-ffff634bb000 1152.0 MB
ffff6b4bc000-ffff734bd000 128.0 MB
ffff834bf000-ffff8b4c0000 128.0 MB

after_alloc_64: 64MB复用2/3号合并空洞的高地址端
ffff1b4b9000-ffff634bb000 1152.0 MB
ffff6b4bc000-ffff734bd000 128.0 MB
ffff7f4be000-ffff8b4c0000 192.0 MB

图 9 maps 关键匿名映射片段

从 maps 片段可以看到, 六个 128MB 块最初被合并为一个 768MB 匿名 VMA。释放 2 号、3 号、5 号后, VMA 被切分为三个仍被占用的 128MB 段, 中间形成 128MB 和 256MB 两类空洞。1024MB 申请无法放入这些空洞, 因而被安排到更低地址区域, 并与低地址端的 6 号块合并为 1152MB 映射。64MB 申请实测地址为 0xffff7f4be010, 落在 2 号与 3 号合并空洞的高地址端, 随后与 1 号块合并为 192MB 映射。这个结果说明我的预测方向正确: 小块会优先复用已有空洞, 但具体落点遵循内核自高向低查找空闲区间的策略。

3. 最大虚拟内存空间测试

3.1 指数扩张与二分搜索代码

max_vmem.c 没有触碰映射页, 而是使用 PROT_NONE 与 MAP_NORESERVE 创建纯虚拟保留区。程序先从 1GB 开始指数翻倍寻找失败上界, 再在成功下界与失败上界之间按页大小二分, 从而得到单个进程一次连续匿名映射能够成功建立的最大长度。

核心代码如下：

```
static int try_map(unsigned long long bytes) {
    void *p = mmap(NULL, bytes, PROT_NONE,
                    MAP_PRIVATE | MAP_ANONYMOUS | MAP_NORESERVE, -1, 0);
    if (p == MAP_FAILED) return 0;
    munmap(p, bytes);
    return 1;
}

while (high < cap && try_map(high)) {
    low = high;
    high <<= 1;
}
while (high - low > page) {
    unsigned long long mid = low + (high - low) / 2;
    mid -= mid % page;
    if (try_map(mid)) low = mid;
    else high = mid;
}
```

图 10 max_vmem 二分搜索核心代码

try_map 只负责验证一个长度是否能建立连续匿名映射，**成功后立刻 munmap 释放**，避免测试过程本身消耗大量地址空间。PROT_NONE 让该区域不可读写，MAP_NORESERVE 避免把虚拟保留误转化为物理承诺。指数扩张阶段快速定位失败上界，二分阶段再按页大小收敛到最大可映射长度，这样既能避免线性试探太慢，也能保证结果粒度与页表管理单位一致。

细节：mid -= mid % page，Linux 的虚拟内存管理以“页”为基本单位，mmap() 的映射长度最终会按页进行管理。如果映射长度不是页大小的整数倍，内核仍会向上对齐到完整页，因此实验中主动进行页对齐，可以避免二分过程中出现非页对齐长度带来的误差

4. 读触页与写触页对比实验

4.1 触页程序核心代码

touch_pages.c 把映射大小、访问模式和保持时间都做成命令行参数。**读模式**对每个页面执行 buf[off]读取，**写模式**对每个页面执行 buf[off]=buf[off]+1。两种模式的虚拟映射长度完全相同，差异只来自读访问和写访问触发的内核页分配路径。

核心代码如下：

```
unsigned char *buf = mmap(NULL, len, PROT_READ | PROT_WRITE,
                           MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
print_status_summary("after mmap before touch");

if (strcmp(mode, "read") == 0) {
    for (size_t off = 0; off < len; off += page) {
        sink ^= buf[off];
    }
} else {
    for (size_t off = 0; off < len; off += page) {
        buf[off] = (unsigned char)(buf[off] + 1);
    }
}
print_usage_delta("touch", &before);
print_status_summary("after touch");
```

图 11 touch_pages 读写触页核心代码

mmap 之后立即打印 status，是为了证明“虚拟区间已建立但尚未触页”时 VmSize 和 VmRSS 会分离。

循环变量 off 每次增加 page，因此每次访问都落到新页的页首；

为什么要有 volatile sink：因为读循环如果只是 buf[off]，编译器可能认为结果没用，把读取优化掉。把值异或进 volatile 变量，可以强迫程序真的读内存；**写分支**执行原地加 1，使每个匿名页必须获得私有物理页。循环结束后再次读取 status 和 getrusage，就能把次缺页、主缺页、RssAnon 和 VmSwap 联系起来分析。

为了让结果能量化，程序在触页前后调用 **getrusage** 统计缺页次数，并用 clock_gettime 计时：

```
struct rusage before;
getrusage(RUSAGE_SELF, &before);
double t0 = now_seconds();

/* read loop or write loop */

double t1 = now_seconds();
print_usage_delta("touch", &before);
printf("touch finished in %.3f seconds\n", t1 - t0);
```

图 12 缺页技术统计和用时统计代码

5. 提高部分实验实现

5.1 双进程物理页竞争设计

提高部分第一项使用 touch_pages 的 **write-hold** 模式完成。A 进程先写入 1792MB 并保持 180 秒，外部进程在 B 运行前读取 /proc/A/status；随后 B 进程写入 4096MB，结束后再次读取 /proc/A/status。由于 B 单独写入容量已经达到虚拟机物理内存规模，内核必须在 A 和 B 的匿名页之间做回收选择。

核心代码如下：

```
/*
 * write-hold 用于双进程竞争实验。
 *
 * A 进程写完不立刻退出，而是 sleep 保持映射；
 * 另一个终端启动 B 进程写入大内存后，可以观察 A 的 VmRSS 下降、
 * VmSwap 上升，从而证明 A 的匿名页被换出。
 */
if (strcmp(mode, "write-hold") == 0 && hold_seconds > 0) {
    printf("holding mapping for %d seconds; inspect /proc/%d/status from another process now.\n",
        hold_seconds, getpid());
    sleep((unsigned int)hold_seconds);
    print_status_summary("after hold");
}
```

图 13 双进程物理页竞争核心代码

5.2 16 页分组访问代码

locality_compare.c 设置两个访问函数，两者总页访问次数完全一致，区别只在工作集是否具有短时间局部性：

- (1) global 模式按“全空间扫一轮、再扫下一轮”的顺序访问
- (2) group16 模式则每次取 16 个连续页，把这些页的重复访问全部完成后再进入下一组。

核心代码如下：

```

static void access_global(unsigned char *buf, size_t pages, size_t
page_size, int repeat) {
    for (int r = 0; r < repeat; ++r)
        for (size_t page = 0; page < pages; ++page)
            buf[page * page_size]++;
}

static void access_group16(unsigned char *buf, size_t pages, size_t
page_size, int repeat) {
    const size_t group = 16;
    for (size_t base = 0; base < pages; base += group) {
        size_t end = base + group;
        if (end > pages) end = pages;
        for (int r = 0; r < repeat; ++r)
            for (size_t page = base; page < end; ++page)
                buf[page * page_size]++;
    }
}

```

图 14 locality_compare 16 页分组访问核心代码

access_global 把 repeat 放在最外层，意味着第 1 轮访问完 4GB 后才会再次回到第 1 页，此时旧页很可能已被换出。access_group16 把 16 个连续页作为一个小工作集，在进入下一组前完成全部重复访问，短时间内反复命中的页面只有 64KB 左右。两个函数都执行 buf[page * page_size]++，因此访问总次数和写入粒度相同，最终时间差异主要来自页面置换压力而不是代码工作量差异。

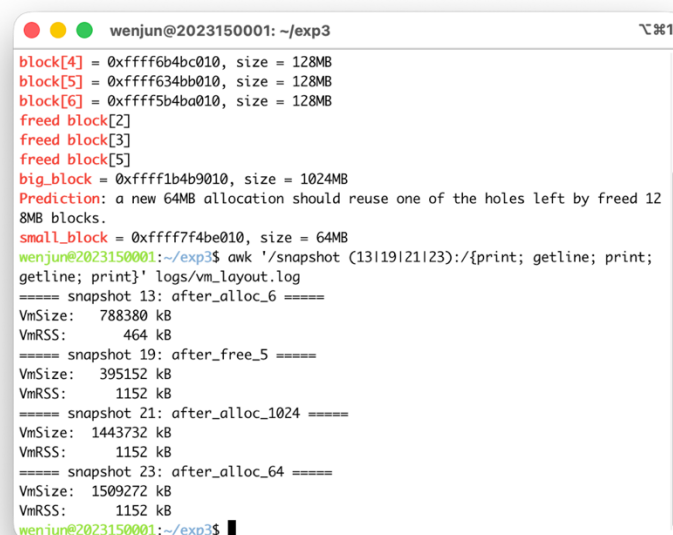
6. 机制横向对比总结

- (1) **从数据介质看：**VmSize 描述的是虚拟地址区间，VmRSS 描述的是当前驻留在物理内存中的页，VmSwap 描述的是被换出到交换区的匿名页。单看 malloc 或 mmap 返回值无法证明物理内存是否真的被占用，必须把这三个字段合起来看。
- (2) **从触发路径看：**读匿名页主要经过缺页异常映射到全局只读零页，写匿名页则必须分配私有物理页并把 PTE 改为可写，因此只有写触页才会让 RssAnon 快速升高。
- (3) **从调度敏感度看：**单进程虚拟映射只受地址空间布局和 VMA 策略影响，双进程密集写入则会触发页面回收和 swap；访问模式虽然不改变访问总量，却会改变页面在再次访问前是否仍留在工作集中。

四、实验结论：（提供运行结果，对结果进行探讨、分析、评价，并提出结论性意见和改进想法）

1. 运行结果验证

1.1 虚存分配、释放与 64MB 预测验证



```
wenjun@2023150001: ~/exp3
block[4] = 0xfffff6b4bc010, size = 128MB
block[5] = 0xfffff634bb010, size = 128MB
block[6] = 0xfffff5b4ba010, size = 128MB
freed block[2]
freed block[3]
freed block[5]
big_block = 0xfffff1b4b9010, size = 1024MB
Prediction: a new 64MB allocation should reuse one of the holes left by freed 12
8MB blocks.
small_block = 0xfffff7f4be010, size = 64MB
wenjun@2023150001:~/exp3$ awk '/snapshot (13|19|21|23):/{print; getline; print;
getline; print}' logs/vm_layout.log
===== snapshot 13: after_alloc_6 =====
VmSize: 788380 kB
VmRSS: 464 kB
===== snapshot 19: after_free_5 =====
VmSize: 395152 kB
VmRSS: 1152 kB
===== snapshot 21: after_alloc_1024 =====
VmSize: 1443732 kB
VmRSS: 1152 kB
===== snapshot 23: after_alloc_64 =====
VmSize: 1509272 kB
VmRSS: 1152 kB
wenjun@2023150001:~/exp3$
```

图 15 终端显示虚存分配实验原始日志

运行结果显示，六次 128MB 申请后 VmSize 从 1924KB 增长到 788380KB，释放 2 号、3 号、5 号后回落到 395152KB。1024MB 申请后 VmSize 升至 1443732KB，64MB 申请后升至 1509272KB。64MB 实测地址 0xfffff7f4be010 位于原 2 号和 3 号释放后形成的高地址空洞内，验证了实验前关于“优先复用已有足够大空洞”的判断。

用户进程空间分配在虚拟地址视角上属于离散分配。每个 VMA 可以连续，但多个 VMA 之间可以被库、栈、vvar、vdso 和已释放空洞隔开。

就本次 mmap 大块分配行为看，内核更接近自高地址向低地址扫描的首次适应，而不是为了找到最小剩余空间而全局搜索的最佳适应。用户空间存在外部碎片，但虚拟地址空间很大，且页表映射把虚拟连续性和物理连续性解耦，所以这种碎片通常不会像传统连续物理分配那样立即导致严重失败。

1.2 单进程最大虚拟空间验证


```
wenjun@2023150001: ~/exp3
wenjun@2023150001:~/exp3$ cat logs/max_vmem.log
RLIMIT_AS: unlimited
single mmap 1.00 GiB: success
single mmap 2.00 GiB: success
single mmap 4.00 GiB: success
single mmap 8.00 GiB: success
single mmap 16.00 GiB: success
single mmap 32.00 GiB: success
single mmap 64.00 GiB: success
single mmap 128.00 GiB: success
single mmap 256.00 GiB: success
single mmap 512.00 GiB: success
single mmap 1024.00 GiB: success
single mmap 2048.00 GiB: success
single mmap 4096.00 GiB: success
single mmap 8192.00 GiB: success
single mmap 16384.00 GiB: success
single mmap 32768.00 GiB: success
single mmap 65536.00 GiB: success
single mmap 131072.00 GiB: success
first failed upper bound: 262144.00 GiB
max contiguous anonymous virtual mapping: 174763.53 GiB (170.67 TiB)
errno after final search step may be stale; result is based on mmap success/failure.
wenjun@2023150001:~/exp3$
```

图 16 终端显示最大虚拟映射测试原始输出

测试机 RLIMIT_AS 显示为 unlimited, 程序从 1GB 一直指数扩张到 128TiB 仍能成功, 首次失败上界为 256TiB, 最终二分得到最大单次连续匿名虚拟映射约 174763.53GiB, 即 170.67TiB。

该值远大于物理内存 3.8GiB, 说明这里测到的是 64 位进程用户地址空间的可映射边界, 而不是实际 DRAM 容量。

1.3 读触页与写触页验证

```
wenjun@2023150001: ~/exp3
wenjun@2023150001:~/exp3$ for f in touch_read_3072 touch_write_3072; do echo ===
== $f ===; sed -n '/touch finished/,/VmSwap:/p' logs/$f.log; echo; done
===== touch_read_3072 =====
touch finished in 0.331 seconds, sink=0
touch page faults: minor +786434, major +0

===== after touch =====
VmSize: 3147652 kB
VmRSS: 1212 kB
RssAnon: 80 kB
RssFile: 1132 kB
VmSwap: 0 kB

===== touch_write_3072 =====
touch finished in 3.435 seconds, sink=0
touch page faults: minor +1572998, major +11

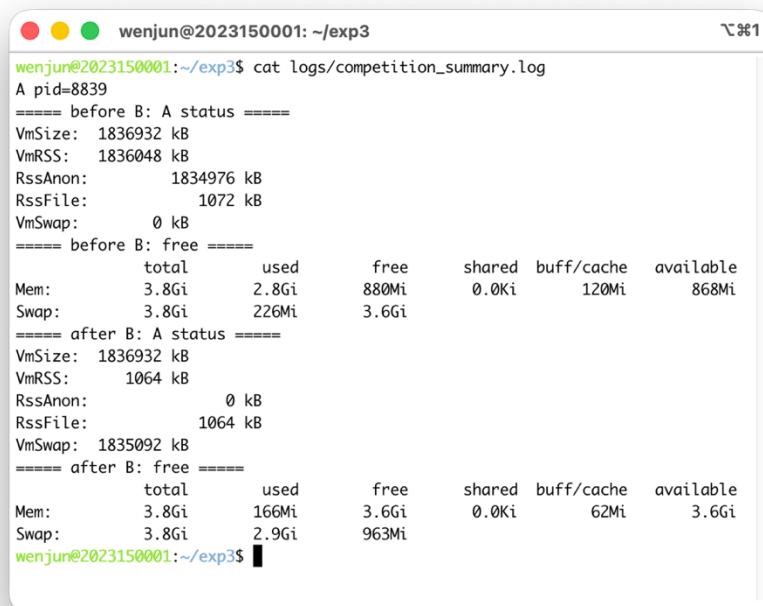
===== after touch =====
VmSize: 3147652 kB
VmRSS: 2750784 kB
RssAnon: 2749572 kB
RssFile: 1212 kB
VmSwap: 396160 kB
wenjun@2023150001:~/exp3$
```


图 17 终端显示 3GB 读触页与写触页状态对比

3GB 读触页后 VmSize 保持 3147652KB, 但 VmRSS 仅约 1212KB, RssAnon 约 80KB, VmSwap 为 0; **这说明读操作虽然触发了 786434 次次缺页, 但基本没有分配私有物理页。**

3GB 写触页后 VmRSS 升至 2750784KB, RssAnon 升至 2749572KB, 同时 VmSwap 达到 396160KB, **证明写访问才会把虚拟页真正落实为匿名物理页, 并在物理内存不足时推动一部分页进入交换区。**

1.4 双进程竞争验证

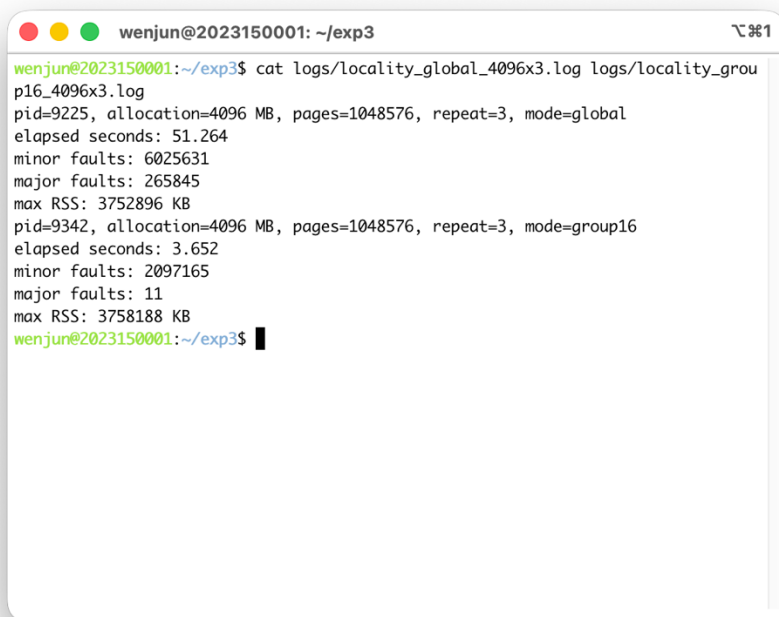


```
wenjun@2023150001: ~/exp3
wenjun@2023150001:~/exp3$ cat logs/competition_summary.log
A pid=8839
===== before B: A status =====
VmSize: 1836932 kB
VmRSS: 1836048 kB
RssAnon: 1834976 kB
RssFile: 1072 kB
VmSwap: 0 kB
===== before B: free =====
Mem: total used free shared buff/cache available
      3.8Gi 2.8Gi 880Mi 0.0Ki 120Mi 868Mi
Swap: 3.8Gi 226Mi 3.6Gi
===== after B: A status =====
VmSize: 1836932 kB
VmRSS: 1064 kB
RssAnon: 0 kB
RssFile: 1064 kB
VmSwap: 1835092 kB
===== after B: free =====
Mem: total used free shared buff/cache available
      3.8Gi 166Mi 3.6Gi 0.0Ki 62Mi 3.6Gi
Swap: 3.8Gi 2.9Gi 963Mi
wenjun@2023150001:~/exp3$
```

图 18 终端显示两个进程竞争物理页运行结果

A 进程写入并保持 1792MB 后, 运行 B 进程前 A 的 VmRSS 为 1836048KB, VmSwap 为 0. B 进程写入 4096MB 后, A 的 VmRSS 被压缩到 1064KB, VmSwap 升到 1835092KB。这个结果非常直接地说明, 当 B 的大工作集进入内存时, 内核把 A 进程暂时不活跃的匿名页批量换出, 两个进程之间确实发生了物理页竞争。

1.5 访问局部性验证



```
wenjun@2023150001: ~/exp3
wenjun@2023150001:~/exp3$ cat logs/locality_global_4096x3.log logs/locality_group16_4096x3.log
pid=9225, allocation=4096 MB, pages=1048576, repeat=3, mode=global
elapsed seconds: 51.264
minor faults: 6025631
major faults: 265845
max RSS: 3752896 KB
pid=9342, allocation=4096 MB, pages=1048576, repeat=3, mode=group16
elapsed seconds: 3.652
minor faults: 2097165
major faults: 11
max RSS: 3758188 KB
wenjun@2023150001:~/exp3$
```

图 19 终端显示全空间循环与 16 页分组访问结果

全空间逐页循环在 4096MB、重复 3 遍的设置下耗时 51.264 秒，主缺页为 265845 次；16 页分组访问在总页访问次数不变的条件下仅耗时 3.652 秒，主缺页只有 11 次。两者差异说明，页面置换的性能损耗不只由访问总量决定，更由下一次访问发生时该页是否仍处在工作集中决定。16 个连续页为一组时，组内 64KB 工作集能够在短时间内保持驻留，因此重复访问几乎不需要从 swap 换回。

2. 结论性评价

综合五组观测，本次实验在多个层面验证了 Linux 虚拟内存子系统的实际行为，主要结论与可改进方向归纳如下。

- (1) **VMA 离散管理**：Linux 进程的用户态虚拟地址空间由一组 VMA 描述，大块 malloc 会被 glibc 走 mmap 路径变成独立匿名 VMA，free 时则按 VMA 边界拆分或归还，maps 中看到的合并与空洞正是这一机制的直接体现。
- (2) **虚拟≠物理**：mmap 或大块 malloc 返回成功只意味着 VMA 登记完成，物理页框尚未分配；只有真正访问到某页时按需分页才会兑现，这一点在 3GB 读触页 VmSize 与 RssAnon 极度分离的现象中得到了量化证据。
- (3) **写触页才走 COW**：匿名页的首次读访问被映射到全局零页，因此 RssAnon 几乎不增长；首次写访问触发 copy-on-write 分配私有物理页，因此写触页才会把 RssAnon 真正抬起来，实验数据显示 3GB 写触页 RssAnon 接近 2.7GB。
- (4) **内存压力下的页面置换**：当多进程匿名工作集之和超过物理内存，内核回收路径会把不活跃匿名页写入 swap；双进程竞争实验中 A 进程的 VmRSS 从约 1.79GB 跌到约 1MB，VmSwap 同步攀升，证明

物理页被回收并转交给 B 进程使用。

- (5) 局部性主导性能：**访问总量相同条件下，仅把访问顺序从全空间循环改为 16 页一组就使运行时间从 51.264 秒下降到 3.652 秒、主缺页从 265845 次降到 11 次，工作集是否能保持驻留比访问总量对性能影响更大。

在以上结论基础上，结合实验过程中暴露出的局限，提出如下可继续推进的方向：

- (1) 扩展至多种页面策略对比：**当前实验默认 Linux 缺省的 overcommit 与回收策略，后续可以改写 `/proc/sys/vm/overcommit_memory`、`/proc/sys/vm/swappiness` 等参数重跑实验，量化策略选择对换页压力的影响。
- (2) 引入透明大页与 madvise 优化：**locality_compare 当前依赖默认 4KB 页，后续可补充开启 THP 或对工作集调用 `madvise(MADV_WILLNEED/MADV_DONTNEED)` 的对照组，进一步分离硬件、内核策略与算法局部性三类因素对性能的贡献。

五、实验体会：（根据自己情况填写）

最开始做这个实验时，我对“分配了内存”这句话的理解仍然有些粗糙，潜意识里总觉得 `malloc` 或 `mmap` 成功后，内核就已经把对应大小的物理内存交给了进程。真正跑完 3GB 读触页后，这个认知被数据直接推翻了：`VmSize` 已经扩张到 3GB 级别，但 `RssAnon` 几乎没有变化。这个结果让我重新把虚拟地址、页表项和物理页三者拆开理解，也意识到操作系统课堂上讲的按需分页并不是抽象概念，而是可以在 `/proc` 文件中被逐项看见的真实机制。

调试过程中最有代表性的坑出现在双进程竞争实验里。我一开始准备用 A 进程自己的输出日志判断竞争前后的 RSS 变化，但 A 进程被重定向到文件后 `stdout` 进入全缓冲状态，中间状态并不能及时落盘。后来我改为在 B 进程运行前后由外部直接读取 `/proc/A/status`，这才拿到 A 的 `VmRSS` 从约 1.79GB 跌到约 1MB、`VmSwap` 升到约 1.75GB 的关键证据。这个小问题提醒我，系统实验不能只相信程序自己的叙述，应该尽量从内核暴露的状态接口交叉验证。

局部性对比实验给我的冲击也很强。同样是 4096MB 空间、同样重复访问 3 遍，只是把访问顺序从全空间循环改成 16 页一组，时间就从 51 秒降到 3 秒多。这个差异让我更具体地理解了为什么操作系统教材会如此强调工作集模型：性能问题并不总是来自“做了多少工作”，也可能来自“工作以什么顺序抵达内存系统”。通过这次实验，我对虚拟内存、物理页、swap 和页面置换之间的关系有了更底层也更工程化的认识。

指导教师批阅意见：

成绩评定：

指导教师签字：

年 月 日

备注：